Running SU2 in parallel: how we set it up

Mayank Dixit, CMU Design Research Collective, 5 October 2026. Every number below was measured on our lab server on that date.

The problem

Our finest CFD meshes (13 to 21 million cells) took 6 to 9 hours per solve. The cause was not the physics: every solve ran as **one process on one core** of a 12-core (24-thread) server.

What we found first

1. **The downloaded "MPI" release did not run in parallel.** The official `SU2-v8.1.0-linux64-mpi` binary on the server is statically linked. Under the system `mpirun -np 8` it started **8 independent copies of the same case** (8 solver banners in the log), and finished slower than one copy (60 s vs 45 s for 30 iterations). It looks like it works, and it doesn't.
2. **So we built SU2 from source against the server's own Open MPI.**

The build (verified, about 10 minutes)

```
mkdir -p ~/build && cd ~/build
git clone --depth 1 --branch v8.4.0 https://github.com/su2code/SU2.git
cd SU2
export CC=mpicc CXX=mpicxx
./meson.py setup build -Dwith-mpi=enabled -Denable-autodiff=false \
  --buildtype=release -Dcpp_args="-march=native" \
  --prefix=$HOME/su2-8.4.0-mpi
./ninja -C build -j 10 install
```

Requirements: `mpicc`/`mpicxx` (Open MPI 4.1.2 here), `gcc`, `git`, Python 3. `-march=native` vectorises for the build machine's CPU; drop it if the binary must run on other CPUs.

Check that the binary really uses MPI:

```
ldd ~/su2-8.4.0-mpi/bin/SU2_CFD | grep libmpi     # must print a libmpi line
```

Two Open MPI pitfalls we hit

Symptom	Cause	Fix
Solver aborts at start with `MPI_Win_create ... invalid window`, even with `mpirun -np 1`	Open MPI 4.1's UCX one-sided component	add `--mca osc ^ucx` to `mpirun` (point-to-point traffic is unaffected)
`mpirun -np 24` refused with "not enough slots"	Open MPI counts physical cores (12), not threads (24)	use at most the physical core count

How the pipeline uses it

The CFD server (`su2-mcp`, file `src/su2_mcp/mpi.py`) decides how to launch every solve, and records the decision in the run's metadata (`launch_mode`, `mpi_ranks`, `launch_reason`):

- It launches in parallel **only when it can prove the binary is MPI-capable**: either an `SU2_CFD_MPI` binary exists, or `SU2_CFD` links an MPI library. Otherwise it runs serially and says why. It never runs copies of a serial binary, because that silently solves the same case N times.

- MPI builds always start through `mpirun`, even for one process.
- Default process count: the machine's physical cores. Override with `SU2_MPI_RANKS=N` (1 forces one process).
- With Open MPI it adds `--mca osc ^ucx` automatically. Override all extra launcher flags with `SU2_MPIRUN_ARGS="..."` (an empty value disables them).

To use the parallel build, put it first on `PATH`:

```
export PATH="$HOME/su2-8.4.0-mpi/bin:$PATH"
```

Results

Speed. D150 airliner, 1.04 M cells, 40 iterations. The server was shared with other users' jobs (load average 17 to 23 of 24 threads), so these are real-world numbers, not best case.

Run	Wall time	Speed-up vs. the old 1-process runs
Old 8.1.0 release binary, 1 process (how our long runs were made)	200.7 s	1.0x
New 8.4.0 build, 1 process	208.9 s	1.0x
New build, 4 processes	57.9 s	3.5x
New build, 8 processes	35.8 s	5.6x
New build, 12 processes	29.6 s	6.8x

Same answer. D150, 343 k cells, solved to convergence (the solver's own stopping criterion on lift):

Run	CL	CD	Iterations	Wall time
1 process	0.3358549	0.0192869	130	203.1 s
12 processes	0.3358560	0.0192870	151	33.3 s

Lift and drag agree to a few parts per million; the parallel run needs a few more iterations (the domain split changes the convergence path slightly) and is still 6.1x faster. Before full convergence the two can differ in the third or fourth digit, so compare converged runs only.

One thing this exposed. On the same mesh and configuration, the old 8.1.0 binary gave CD 0.02076 and 8.4.0 gives 0.01929 (CL agrees within 0.03 %). Drag values carry a solver-version sensitivity of about 7 % on this case; record the SU2 version with every result.

Checklist for a new machine

1. Build as above (or confirm an existing binary with `ldd ... | grep libmpi`).
2. Run any case with `mpirun --mca osc ^ucx -np 4 SU2_CFD case.cfg` and check the log shows **one** SU2 banner, not four.
3. Solve one case to convergence on 1 and on N processes and confirm CL and CD agree.
4. Put the build first on `PATH`; the pipeline picks it up automatically.

Not yet done: a macOS build. The same recipe should work with Homebrew's `open-mpi`, but we have not verified it.